Security in Odoo

Aside from manually managing access using custom code, Odoo provides two main data-driven mechanisms to manage or restrict access to data.

Both mechanisms are linked to specific users through *groups*: a user belongs to any number of groups, and security mechanisms are associated to groups, thus applying security mechanisms to users.

class **res.groups**

name

serves as user-readable identification for the group (spells out the role / purpose of the group)

category_id

The *module category*, serves to associate groups with an Odoo App (~a set of related business models) and convert them into an exclusive selection in the user form.

implied_ids

Other groups to set on the user alongside this one. This is a convenience pseudo-inheritance relationship: it's possible to explicitly remove implied groups from a user without removing the implier.

comment

Additional notes on the group e.g.

Access Rights

Grants access to an entire model for a given set of operations. If no access rights matches an operation on a model for a user (through their group), the user doesn't have access.

Access rights are additive, a user's accesses are the union of the accesses they get through all their groups e.g. given a user who is part of group A granting read and create access and a group B granting update access, the user will have all three of create, read, and update.

class **ir.model.access**

name

The model whose access the ACL controls.

group_id

The `res.groups` to which the accesses are granted, an empty `group_id` means the ACL is granted to *every user* (non-employees e.g. portal or public users).

The `perm_{method}` attributes grant the corresponding CRUD access when set, they are all unset by default.

perm_create

perm_read

perm_write

perm_unlink

Record Rules

Record rules are *conditions* which must be satisfied in order for an operation to be allowed. Record rules are evaluated record-by-record, following access rights.

Record rules are default-allow: if access rights grant access and no rule applies to the operation and model for the user, the access is granted.

***class* ir.rule**

name

The description of the rule.

model_id

The model to which the rule applies.

groups

The `res.groups` to which access is granted (or not). Multiple groups can be specified. If no group is specified, the rule is *global* which is treated differently than "group" rules (see below).

global

domain_force

A predicate specified as a **domain**, the rule allows the selected operations if the domain matches the record, and forbids it otherwise.

The domain is a *python expression* which can use the following variables:

time
Python's `time` module.

user
The current user, as a singleton recordset.

company_id
The current user's currently selected company as a single company id (not a recordset).

company_ids
All the companies to which the current user has access as a list of company ids (not a recordset), see **Security rules** for more details.

The `perm_{method}` have completely different semantics than for `ir.model.access` : for rules, they specify which operation the rules applies *for*. If an operation is not selected, then the rule is not checked for it, as if the rule did not exist.

All operations are selected by default.

perm_create

perm_read

perm_write

perm_unlink

Global rules versus group rules

There is a large difference between global and group rules in how they compose and combine:

- Global rules *intersect*, if two global rules apply then *both* must be satisfied for the access to be granted, this means adding global rules always restricts access further.
- Group rules *unify*, if two group rules apply then *either* can be satisfied for the access to be granted. This means adding group rules can expand access, but not beyond the bounds defined by global rules.

odoo docs

> **Danger**
>
> Creating multiple global rules is risky as it's possible to create non-overlapping rulesets, which will remove all access.

Field Access

An ORM `Field` can have a `groups` attribute providing a list of groups (as a comma-separated string of **external identifiers**).

If the current user is not in one of the listed groups, he will not have access to the field:

- restricted fields are automatically removed from requested views
- restricted fields are removed from `fields_get()` responses
- attempts to (explicitly) read from or write to restricted fields results in an access error

Security Pitfalls

As a developer, it is important to understand the security mechanisms and avoid common mistakes leading to insecure code.

Unsafe Public Methods

Any public method can be executed via a **RPC call** with the chosen parameters. The methods starting with a `_` are not callable from an action button or external API.

On public methods, the record on which a method is executed and the parameters can not be trusted, ACL being only verified during CRUD operations.

```
# this method is public and its arguments can not be trusted
def action_done(self):
    if self.state == "draft" and self.env.user.has_group('base.manager'):
        self._set_state("done")

# this method is private and can only be called from other python methods
def _set_state(self, new_state):
    self.sudo().write({"state": new_state})
```

Making a method private is obviously not enough and care must be taken to use it properly.

translations, invalidation of fields, `active`, access rights and so on.

And chances are that you are also making the code harder to read and probably less secure.

```
# very very wrong
self.env.cr.execute('SELECT id FROM auction_lots WHERE auction_id in (' + ','.join(
auction_lots_ids = [x[0] for x in self.env.cr.fetchall()]

# no injection, but still wrong
self.env.cr.execute('SELECT id FROM auction_lots WHERE auction_id in %s '\
           'AND state=%s AND obj_price > 0', (tuple(ids), 'draft',))
auction_lots_ids = [x[0] for x in self.env.cr.fetchall()]

# nearly there
auction_lots_ids = [x[x] for x in self.env.execute_query(SQL("""
    SELECT id FROM auction_lots
    WHERE auction_id IN %s AND state = %s AND obj_price > 0
""", tuple(ids), 'draft'))

# better
auction_lots_ids = self.search([('auction_id','in',ids), ('state','=','draft'), ('o
```

SQL injections

Care must be taken not to introduce SQL injections vulnerabilities when using manual SQL queries. The vulnerability is present when user input is either incorrectly filtered or badly quoted, allowing an attacker to introduce undesirable clauses to a SQL query (such as circumventing filters or executing `UPDATE` or `DELETE` commands).

The best way to be safe is to never, NEVER use Python string concatenation (+) or string parameters interpolation (%) to pass variables to a SQL query string.

The second reason, which is almost as important, is that it is the job of the database abstraction layer (psycopg2) to decide how to format query parameters, not your job! For example psycopg2 knows that when you pass a list of values it needs to format them as a comma-separated list, enclosed in parentheses !

Even better, there exists a `SQL` wrapper to build queries using templates that handles the formatting of inputs. Check **SQL execution** for detailed usage.

```
#   - it's not your job to format the list of ids
self.env.cr.execute('SELECT distinct child_id FROM account_account_consol_rel ' +
           'WHERE parent_id IN ('+','.join(map(str, ids))+')')

# better
self.env.cr.execute('SELECT DISTINCT child_id '\
           'FROM account_account_consol_rel '\
           'WHERE parent_id IN %s',
           (tuple(ids),))

# more readable
self.env.cr.execute(SQL("""
    SELECT DISTINCT child_id
    FROM account_account_consol_rel
    WHERE parent_id IN %s
""", tuple(ids)))
```

This is very important, so please be careful also when refactoring, and most importantly do not copy these patterns!

Here is a memorable example to help you remember what the issue is about (but do not copy the code there). Before continuing, please be sure to read the online documentation of pyscopg2 to learn of to use it properly:

- The problem with query parameters
- How to pass parameters with psycopg2
- Advanced parameter types
- Psycopg documentation

Building the domains

Domains are represented as lists and are serializable. You would be tented to manipulate these lists directly, however it can introduce subtle issues where the user could inject a domain if the input is not normalized. Use **`Domain`** to handle safely the manipulation of domains.

```
# bad
# the user can just pass ['|', ('id', '>', 0)] to access all
domain = ...  # passed by the user
security_domain = [('user_id', '=', self.env.uid)]
domain += security_domain  # can have a side-effect if this is a function argument
```

```
domain &= Domain('user_id', '=', self.env.uid)
self.search(domain)
```

Unescaped field content

When rendering content using JavaScript and XML, one may be tempted to use a `t-raw` to display rich-text content. This should be avoided as a frequent **XSS** vector.

It is very hard to control the integrity of the data from the computation until the final integration in the browser DOM. A `t-raw` that is correctly escaped at the time of introduction may no longer be safe at the next bugfix or refactoring.

```
QWeb.render('insecure_template', {
    info_message: "You have an <strong>important</strong> notification",
})
```

```
<div t-name="insecure_template">
    <div id="information-bar"><t t-raw="info_message" /></div>
</div>
```

The above code may feel safe as the message content is controlled but is a bad practice that may lead to unexpected security vulnerabilities once this code evolves in the future.

```
// XSS possible with unescaped user provided content !
QWeb.render('insecure_template', {
    info_message: "You have an <strong>important</strong> notification on " \
        + "the product <strong>" + product.name + "</strong>",
})
```

While formatting the template differently would prevent such vulnerabilities.

```
QWeb.render('secure_template', {
    message: "You have an important notification on the product:",
```

```
<div t-name="secure_template">
    <div id="information-bar">
        <div class="info"><t t-esc="message" /></div>
        <div class="subject"><t t-esc="subject" /></div>
    </div>
</div>
```

```
.subject {
    font-weight: bold;
}
```

Creating safe content using `Markup`

See the official documentation for explanations, but the big advantage of `Markup` is that it's a very rich type overrinding `str` operations to *automatically escape parameters*.

This means that it's easy to create *safe* html snippets by using `Markup` on a string literal and "formatting in" user-provided (and thus potentially unsafe) content:

```
>>> Markup('<em>Hello</em> ') + '<foo>'
Markup('<em>Hello</em> &lt;foo&gt;')
>>> Markup('<em>Hello</em> %s') % '<foo>'
Markup('<em>Hello</em> &lt;foo&gt;')
```

though it is a very good thing, note that the effects can be odd at times:

```
>>> Markup('<a>').replace('>', 'x')
Markup('<a>')
>>> Markup('<a>').replace(Markup('>'), 'x')
Markup('<ax')
>>> Markup('<a&gt;').replace('>', 'x')
Markup('<ax')
>>> Markup('<a&gt;').replace('>', '&')
Markup('<a&amp;')
```

odoo docs

The **escape** method (and its alias **html_escape**) turns a `str` into a **Markup** and escapes its content. It will not escape the content of a **Markup** object.

```
def get_name(self, to_html=False):
    if to_html:
        return Markup("<strong>%s</strong>") % self.name  # escape the name
    else:
        return self.name

>>> record.name = "<R&D>"
>>> escape(record.get_name())
Markup("&lt;R&amp;D&gt;")
>>> escape(record.get_name(True))
Markup("<strong>&lt;R&amp;D&gt;</strong>")  # HTML is kept
```

When generating HTML code, it is important to separate the structure (tags) from the content (text).

```
>>> Markup("<p>") + "Hello <R&D>" + Markup("</p>")
Markup('<p>Hello &lt;R&amp;D&gt;</p>')
>>> Markup("%s <br/> %s") % ("<R&D>", Markup("<p>Hello</p>"))
Markup('&lt;R&amp;D&gt; <br/> <p>Hello</p>')
>>> escape("<R&D>")
Markup('&lt;R&amp;D&gt;')
>>> _("List of Tasks on project %s: %s",
...     project.name,
...     Markup("<ul>%s</ul>") % Markup().join(Markup("<li>%s</li>") % t.name for t
... )
Markup('Liste de tâches pour le projet &lt;R&amp;D&gt;: <ul><li>First &lt;R&amp;D&g

>>> Markup("<p>Foo %</p>" % bar)  # bad, bar is not escaped
>>> Markup("<p>Foo %</p>") % bar  # good, bar is escaped if text and kept if markup

>>> link = Markup("<a>%s</a>") % self.name
>>> message = "Click %s" % link  # bad, message is text and Markup did nothing
>>> message = escape("Click %s") % link  # good, format two markup objects together
```

odoo docs

When working with translations, it is especially important to separate the HTML from the text. The translation methods accepts a **`Markup`** parameters and will escape the translation if it gets receives at least one.

```
>>> Markup("<p>%s</p>") % _("Hello <R&D>")
Markup('<p>Bonjour &lt;R&amp;D&gt;</p>')
>>> _("Order %s has been confirmed", Markup("<a>%s</a>") % order.name)
Markup('Order <a>SO42</a> has been confirmed')
>>> _("Message received from %(name)s <%(email)s>",
...   name=self.name,
...   email=Markup("<a href='mailto:%s'>%s</a>") % (self.email, self.email)
Markup('Message received from Georges &lt;<a href=mailto:george@abitbol.example>geo
```

Escaping vs Sanitizing

> **Important**
>
> Escaping is always 100% mandatory when you mix data and code, no matter how safe the data

Escaping converts *TEXT* to *CODE*. It is absolutely mandatory to do it every time you mix *DATA/TEXT* with *CODE* (e.g. generating HTML or python code to be evaluated inside a `safe_eval`), because *CODE* always requires *TEXT* to be encoded. It is critical for security, but it's also a question of correctness. Even when there is no security risk (because the text is 100% guarantee to be safe or trusted), it is still required (e.g. to avoid breaking the layout in generated HTML).

Escaping will never break any feature, as long as the developer identifies which variable contains *TEXT* and which contains *CODE*.

```
>>> from odoo.tools import html_escape, html_sanitize
>>> data = "<R&D>" # `data` is some TEXT coming from somewhere

# Escaping turns it into CODE, good!
>>> code = html_escape(data)
>>> code
Markup('&lt;R&amp;D&gt;')
```

odoo docs

Sanitizing converts *CODE* to *SAFER CODE* (but not necessary *safe* code). It does not work on *TEXT*. Sanitizing is only necessary when *CODE* is untrusted, because it comes in full or in part from some user-provided data. If the user-provided data is in the form of *TEXT* (e.g. the content from a form filled by a user), and if that data was correctly escaped before putting it in *CODE*, then sanitizing is useless (but can still be done). If however, the user-provided data was **not escaped**, then sanitizing will **not** work as expected.

```
# Sanitizing without escaping is BROKEN: data is corrupted!
>>> html_sanitize(data)
Markup('')

# Sanitizing *after* escaping is OK!
>>> html_sanitize(code)
Markup('<p>&lt;R&amp;D&gt;</p>')
```

Sanitizing can break features, depending on whether the *CODE* is expected to contain patterns that are not safe. That's why `fields.Html` and `tools.html_sanitize()` have options to fine-tune the level of sanitization for styles, etc. Those options have to be carefully considered depending on where the data comes from, and the desired features. The sanitization safety is balanced against sanitization breakages: the safer the sanitisation the more likely it is to break things.

```
>>> code = "<p class='text-warning'>Important Information</p>"
# this will remove the style, which may break features
# but is necessary if the source is untrusted
>>> html_sanitize(code, strip_classes=True)
Markup('<p>Important Information</p>')
```

Evaluating content

Some may want to `eval` to parse user provided content. Using `eval` should be avoided at all cost. A safer, sandboxed, method **`safe_eval`** can be used instead but still gives tremendous capabilities to the user running it and must be reserved for trusted privileged users only as it breaks the barrier between code and data.

```
# better but still not recommended
from odoo.tools import safe_eval
domain = safe_eval(self.filter_domain)
return self.search(domain)

# good
from ast import literal_eval
domain = literal_eval(self.filter_domain)
return self.search(domain)
```

Parsing content does not need `eval`

Language	Data type	Suitable parser
Python	int, float, etc.	int(), float()
Javascript	int, float, etc.	parseInt(), parseFloat()
Python	dict	json.loads(), ast.literal_eval()
Javascript	object, list, etc.	JSON.parse()

Accessing object attributes

If the values of a record needs to be retrieved or modified dynamically, one may want to use the `getattr` and `setattr` methods.

```
# unsafe retrieval of a field value
def _get_state_value(self, res_id, state_field):
    record = self.sudo().browse(res_id)
    return getattr(record, state_field, False)
```

This code is however not safe as it allows to access any property of the record, including private attributes or methods.

The `__getitem__` of a recordset has been defined and accessing a dynamic field value can be easily achieved safely:

```
    return record[state_field]
```

The above method is obviously still too optimistic and additional verifications on the record id and field value must be done.

Get Help

Contact Support | Ask the Odoo Community